Aurora

Towards Universal Generative Multimodal Time Series Forecasting

ICLR 2026

Aurora — Main

The official code repo of our ICLR 26's paper: Aurora: Towards Universal Generative Multimodal Time Series Forecasting

---

Introduction

Aurora is a highly capable multimodal time series foundation model. Based on the **Modality-Guided Multi-head Self-Attention** and **Prototype-Guided Flow Matching**, Aurora can effectively utilize the domain-specific knowledge contained in modalities and support generative probabilistic forecasting, thus covering versatile forecasting scenarios.

See **Figure 1**, to our best knowldege, Aurora is the first pretrained multimodal time series foundation model! Evaluated on **5** well-recognized benchmarks, including TimeMMD, TSFM-Bench, ProbTS, TFB, and EPF, Aurora is demonstrated the state-of-the-art.

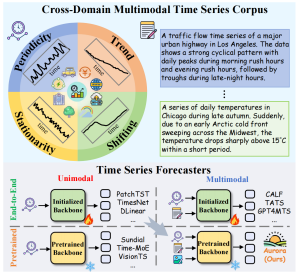


Figure 1: Aurora is pretrained on *cross-domain multimodal* time series corpus, supporting both text and image information to enhance zero-shot time series forecasting.

Architecture

In this work, we pretrain Aurora in a cross-modality paradigm, which adopts Channel-Independence on time series data, and models corresponding multimodal interaction to inject domain knowledge. Note that the each variable of time series is first normalized through Instance Normalization to mitigate the value discrepancy. See **Figure 2**, Aurora mainly consists of two phases: 1) in Aurora Encoder, we tokenize and encode each modality into modal features, then fuse them to form multimodal representations; 2) in Aurora Decoder, we utilize a Condition Decoder to obtain the multimodal conditions of future tokens, leverage a Prototype Retreiver to retrieve the future prototypes based on the domain knowledge, and conduct flow matching on them to make generative probabilistic forecasts.

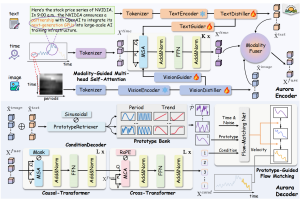


Figure 2: The overview of Aurora.

Quickstart

From pypi (recommended)

We have published Aurora on PyPi, **you can directly install it with one line of code!**

```
$ pip install aurora-model==0.1.0
```

Then you can use the Aurora model to make zero-shot probabilistic forecasting!

Unimodal Time Series Forecasting

```
from aurora import load_model
import os
import torch
# os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'
model = load_model()

# prepare input
batch_size, lookback_length = 1, 528
seqs = torch.randn(batch_size, lookback_length).cuda()

# Note that Aurora can generate multiple probable predictions
forecast_length = 96
num_samples = 100


# For inference_token_len, you can refer to LightGTS (Periodic Patching).
# We recommend to use the period length as the inference_token_len.
output = model.generate(inputs=seqs, max_output_length=forecast_length, num_samples=num_samples, inference_token_len=4


# use raw predictions for mean/quantiles/confidence-interval estimation
print(output.shape)
```

Multimodal Time Series Forecasting

```
from aurora import load_model
from einops import rearrange
import os
import torch
# os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'
model = load_model()
tokenizer = model.tokenizer

# prepare input
batch_size, n_vars, lookback_length, max_text_length = 1, 10, 528, 200
seqs = torch.randn(batch_size, lookback_length, n_vars).cuda()

text = "1983-09-12: The Federal Register provides a uniform system for making available to the public regulations and

tokenized_text = tokenizer(text, padding='max_length', truncation=True, max_length=max_text_length, return_tensors="pt
text_input_ids = tokenized_text['input_ids'].cuda()
text_attention_mask = tokenized_text['attention_mask'].cuda()
text_token_type_ids = tokenized_text.get('token_type_ids', torch.zeros_like(text_input_ids)).cuda()

batch_input_ids = text_input_ids.repeat(n_vars, 1)
batch_attention_mask = text_attention_mask.repeat(n_vars, 1)
batch_token_type_ids = text_token_type_ids.repeat(n_vars, 1)
batch_x = rearrange(seqs, "b l c -> (b c) l")
```

```
# Note that Aurora can generate multiple probable predictions
forecast_length = 96
num_samples = 100


# For inference_token_len, you can refer to LightGTS (Periodic Patching).
# We recommend to use the period length as the inference_token_len.
output = model.generate(inputs=batch_x,text_input_ids=batch_input_ids,
                        text_attention_mask=batch_attention_mask,
                        text_token_type_ids=batch_token_type_ids,
                        max_output_length=forecast_length,
                        num_samples=num_samples,
                        inference_token_len=48)


# use raw predictions for mean/quantiles/confidence-interval estimation
print(output.shape)
```

From raw code

We release the original code of Aurora in this repo. You can also download the pretrained checkpoints in our huggingface repo and put them in the folder: aurora/.

If you want to pretrain an Aurora on your own time series corpus, you need to install the following important packages:

```
$ pip install torch==2.4.0
$ pip install torchvision==0.19.0
$ pip install transformers[torch]

from huggingface_hub import snapshot_download
import os
import torch
# os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'

# --- Configuration ---
repo_id = "DecisionIntelligence/Aurora"

# Target directory for the download. "." represents the current working directory.
local_dir = "./work_dir"

# Optional: Set repo_type to "dataset" or "space" if you are not downloading a model.
repository_type = "model"
# ---------------------

# Ensure the local directory exists before starting the download
if not os.path.exists(local_dir):
    os.makedirs(local_dir)
    print(f"Created directory: {local_dir}")

print(f"Starting download from '{repo_id}' to '{local_dir}'...")

try:
    # snapshot_download handles the download of all files in the repository
    download_path = snapshot_download(
        repo_id=repo_id,
        local_dir=local_dir,
        # Set to False to download actual files instead of symbolic links
        local_dir_use_symlinks=False,
        repo_type=repository_type,
        # Use your HF access token for private/gated repositories
        token=None
    )
    print(f"\nSuccess! All files downloaded to: {download_path}")
except Exception as e:
```

```
    print(f"\nAn error occurred during download: {e}")

# Then you can easily make zero-shot forecasts using Aurora

from modeling_aurora import AuroraForPrediction

model = AuroraForPrediction.from_pretrained("./",trust_remote_code=True)

# prepare input
batch_size, lookback_length = 1, 528
seqs = torch.randn(batch_size, lookback_length).cuda()

# Note that Aurora can generate multiple probable predictions
forecast_length = 96
num_samples = 100


# For inference_token_len, you can refer to LightGTS (Periodic Patching).
# We recommend to use the period length as the inference_token_len.
output = model.generate(inputs=seqs, max_output_length=forecast_length, num_samples=num_samples, inference_token_len=4


# use raw predictions for mean/quantiles/confidence-interval estimation
print(output.shape)
```

Experiments

For benchmarking (TSFM-Bench, ProbTS, TimeMMD, TFB, and EPF), you can install additional packages based on the requirement files under folders, and the datasets can be fetched from this link. All experimental results can be reproduced by running the scripts in the benchmark folder■

```
# TimeMMD
TimeMMD/scripts/run_aurora_timemmd_zero_shot.sh

# EPF
EPF/scripts/run_aurora_short_term_zero_shot.sh

# ProbTS
ProbTS/scripts/run_aurora_probts.sh

# TSFM-Bench
TFB/scripts/run_aurora_tfb.sh

# TFB univariate
TFB/scripts/run_aurora_uni.sh
```

Performance

Aurora ahieves consistent state-of-the-art performance on these 5 benchmarks:

Table 1: Average results of multimodal zero-shot & few-shot forecasting experiments on datasets from TimeMMD. Lower MSE or MAE values indicate better predictions. **Red**: the best, Blue: the 2nd best. All the results are listed in Table 12 of Appendix B.

Type	🌟Zero-shot Foundation Models																							
Models	Aurora (Ours)		Sundial (2025)		Timesfm (2025)		ROSE (2025)		NORM4 (2024)		Informer (Base)		GPT4TS (2023)		TATS (2023)		CALF (2025)		Time-FLM (2025)	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Agriculture	0.777	0.240	0.971	0.302	0.298	0.206	0.345	0.372	0.531	0.469	0.821	0.629	0.329	0.238	0.213	0.198	0.250	0.213	0.327	0.302
Climate	0.843	0.246	0.873	0.267	0.271	0.205	0.335	0.365	0.557	0.487	0.854	0.669	0.313	0.243	0.236	0.224	0.251	0.220	0.319	0.301
Energy	0.615	0.196	0.378	0.151	0.191	0.192	0.187	0.247	0.401	0.452	0.856	0.699	0.242	0.247	0.607	0.606	0.141	0.241	0.854	0.510
Group	1.335	0.372	0.172	0.267	0.384	0.422	0.386	0.479	0.324	0.417	0.126	0.325	0.262	0.380	0.219	0.334	0.104	0.241	0.268	0.379
Environment	0.776	0.272	0.350	0.434	0.555	0.496	0.392	0.696	0.236	0.401	0.269	0.507	0.321	0.480	0.219	0.396	0.325	0.511	0.218	0.387
Health	1.052	0.809	0.092	0.092	1.676	1.232	0.098	1.385	1.776	1.241	1.347	0.776	1.064	0.798	1.196	0.797	1.498	0.772	1.989	0.838
Security	0.452	0.003	0.443	0.005	0.108	0.047	0.0.245	0.760	0.543.245	0.715	0.662	0.566	0.447	0.003	0.386	0.007	0.396	0.007	0.396	0.132
SocialGood	0.838	0.514	1.186	0.572	1.126	0.613	1.141	0.591	1.438	0.841	0.811	0.496	0.910	0.491	0.910	0.492	0.918	0.506	0.908	0.488
Traffic	0.461	0.289	0.421	0.601	0.320	0.407	0.421	0.601	0.406	0.468	0.417	0.490	0.460	0.301	0.438	0.296	0.422	0.295	0.438	0.312
1st Count	0	0	1	1	0	0	0	0	1	0	0	0	0	0	1	1	1	1	0	0

Table 2: Average results of unimodal zero-shot deterministic forecasting experiments on datasets from TSFM-Bench. Lower MSE or MAE values indicate better predictions. (*) denotes datasets included in the model's pretraining and therefore excluded from testing. **Red**: the best, Blue: the 2nd best. All the results are listed in Table 13 of Appendix B.

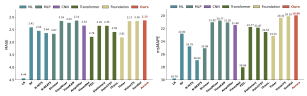
Type	🌟Zero-shot Foundation Models																			
Models	Aurora (Ours)		Sundial (2025)		ROSE (2025)		Timer (2024)		TimesFM (2024)		Chronos (2024)		Time-MoE (2025)		TimeTS (2023)		MOIRAI (2024)		TTM (2024)	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh2	0.378	0.376	0.337	0.370	0.793	0.411	0.371	0.379	0.417	0.406	0.462	0.404	0.577	0.390	0.671	0.417	0.383	0.396	0.441	0.439
Weather	0.230	0.267	0.216	0.270	0.285	0.303	0.292	0.312	-	-	0.288	0.309	0.326	0.336	0.275	0.298	0.268	0.275	0.283	0.307
Electricity	0.176	0.275	0.169	0.268	0.234	0.326	0.297	0.375	-	-	-	-	-	-	0.496	0.291	0.188	0.272	0.222	0.317
Traffic	0.210	0.252	-	-	0.398	0.412	0.415	0.407	-	-	0.615	0.412	-	-	-	-	-	-	0.564	0.396
Artur	0.205	0.295	0.211	0.292	0.464	0.349	0.771	0.460	0.988	0.597	0.391	0.319	0.411	0.328	0.665	0.409	0.714	0.561	0.419	0.318
PEMS08	0.763	0.512	-	-	1.369	0.579	0.866	0.602	1.485	0.682	1.747	1.024	-	-	1.251	0.679	-	-	1.738	1.086
Wiki	7.115	0.762	7.193	0.721	7.237	0.820	7.303	0.783	14.3	0.970	11.475	0.814	-	-	14.5	0.848	2.398	0.795	7.317	0.823
NN5B	0.928	0.726	0.860	0.612	-	-	0.988	0.761	0.823	0.726	1.129	0.720	-	-	1.320	0.830	-	-	-	-
1st Count	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 3: Average results of unimodal zero-shot probabilistic forecasting experiments on datasets from ProbTS. Lower MSE or MAE values indicate better predictions. ('-') denotes datasets included in the model's pretraining and therefore excluded from testing. **Red**: the best, Blue: the 2nd best. All the results are listed in Table 14 of Appendix B.

Type	Zero-shot Foundation Models								Full-shot Probabilistic End-to-end Supervised Models							
Models	**Aurora (Ours)**		**Sundial** (2026)		**TimeCLIP** (2025)		**MOIRAI** (2024)		**Long-Llama** (2025)		**TimeGrad** (2019)		**CSDI** (2021)		**CRIT MAF** (2021)	
Metrics	CRPS	NMAE	CRPS	NMAE	CRPS	NMAE	CRPS	NMAE	CRPS	NMAE	CRPS	NMAE	CRPS	NMAE	CRPS	NMAE
ETT (avg)	0.414	0.397	0.381	0.375	0.396	0.516	0.366	0.377	0.375	0.530	0.579	0.487	0.366	0.500	0.405	0.436
Weather	0.070	0.076	0.087	0.32	0.142	0.178	0.129	0.143	0.096	0.206	0.132	0.154	0.077	0.085	0.125	0.155
Elecnetwork	0.040	0.035	**0.043**	**0.050**	-	-	0.247	0.298	-	-	0.487	0.519	/	/	0.102	0.126
Traffic	0.120	0.163	-	-	0.209	0.295	-	-	0.338	0.385	0.327	0.392	/	/	0.225	0.264
Exchange	0.010	0.007	0.043	0.049	0.010	0.007	0.049	0.036	0.057	0.040	0.088	0.111	0.009	0.006	0.082	0.093
ILI	0.147	0.666	0.148	0.646	0.178	0.197	0.139	0.197	0.156	0.211	0.548	0.599	0.276	0.290	0.266	0.318
1^{st}Count	10	24	5	8	1	1	2	1	0	0	0	0	4	1	1	1

Table 4: Results of short-term zero-shot forecasting experiments on datasets from EPF. Lower MSE or MAE values indicate better predictions. ('-') denotes datasets included in the model's pretraining and therefore excluded from testing. **Red**: the best, Blue: the 2nd best.

Type	Zero-shot Foundation Models								Full-shot End-to-end Supervised Models							
Models	**Aurora (Ours)**		**Sundial** (2025)		**TimesFM** (2025)		**MOSE** (2023)		**MOIRAI** (2024)		**TimeNet** (2024)		**iTransformer** (2024)		**PatchTST** (2023)		**TimesNet** (2023)	
Metrics	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
NP	0.268	0.315	0.336	0.277	0.518	0.461	0.666	0.556	0.668	0.558	**0.238**	**0.305**	0.265	0.318	0.307	0.284	0.280	0.299
PJM	**0.862**	**0.185**	0.888	0.189	0.25	0.366	0.71	0.402	0.736	0.423	0.988	0.395	0.897	0.37	0.106	0.208	0.097	0.197
BE	**0.341**	0.237	0.371	0.270	0.679	0.477	0.815	0.514	0.837	0.534	0.574	**0.241**	0.394	0.279	0.403	0.264	0.419	0.288
FR	0.307	**0.206**	0.302	0.207	0.625	0.393	0.796	0.447	0.751	0.454	**0.301**	0.211	0.439	0.233	0.411	0.228	0.431	0.234
DE	0.539	0.475	0.541	0.484	0.966	0.687	1.276	0.775	1.251	0.779	**0.440**	**0.415**	0.479	0.433	0.84	0.432	0.902	0.446



(a) Mean MASE results. (b) Mean msMAPE results.

Figure 5: Mean MASE and msMAPE results of 8,068 univariate datasets in TFB. The full results can be found in Table 10 and 11 of Appendix B. **Red**: the best.

Citation

If you find this repo useful, please cite our paper.

```
@inproceedings{wu2026aurora,
  title     = {Aurora: Towards Universal Generative Multimodal Time Series Forecasting},
  author    = {Wu, Xingjian and Jin, Jianxin and Qiu, Wanghui and Chen, Peng and Shu, Yang and Yang, Bin and Guo, Chen
  booktitle = {ICLR},
  year      = {2026}
}
```

Contact

If you have any questions or suggestions, feel free to contact:

- Xingjian Wu (xjwu@stu.ecnu.edu.cn)
- Peng Chen (pchen@stu.ecnu.edu.cn)

Or describe it in Issues.

TFB — Characteristics Extractor

1. Environment Setup

• Create a Conda environment with the following specifications: Python 3.8, R 4.3.1.

```
conda create -n char_extractor python=3.8 r-base=4.3.1
conda activate char_extractor
```

• Set environment variables for R:

```
export R_HOME=$CONDA_PREFIX/lib/R
export PATH=$PATH:$R_HOME/bin
```

• Install necessary R packages:

```
conda install -c https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge r-tidyverse r-Rcatch22 r-forecast r-t
```

• Install required Python packages:

```
pip install rpy2==3.5.16 pandas==1.5.3 scipy==1.10.1 numpy==1.24.4 statsmodels==0.14.1 scikit_learn==1.3.2
```

2. Input

• Data Format

The input data format needs to be TFB, supporting a three-column long table format. For more details, please refer to here.

• Input Format

• You can provide the path to a single time series file, and the tool will compute the characteristics of that time series. For example: ./DemoDatasets/Exchange.csv.

• Alternatively, you can specify a folder path, and the tool will recursively calculate the time series features for all files within the folder. For example: ./DemoDatasets.

3. Output

• Output Format

• For univariate time series files, for example, if the input file is named m4_hourly_dataset_69.csv, two corresponding files will be generated in the user-specified output directory (default is the "characteristics" folder):

• All_characteristics_m4_hourly_dataset_69.csv: Contains all time series features calculated by this code.

• TFB_characteristics_m4_hourly_dataset_69.csv: Contains the time series features used in the TFB paper.

• For multivariate time series files, for example, if the input file is named Exchange.csv, four corresponding files will be generated in the user-specified output directory (default is the "characteristics" folder):

• All_characteristics_Exchange.csv: Contains all time series features calculated by this code. Each row corresponds to the time series features of one variable in the multivariate series.

• TFB_characteristics_Exchange.csv: Contains the time series features used in the TFB paper. Each row corresponds to the time series features of one variable in the multivariate series.

• mean_All_characteristics_Exchange.csv: Contains all time series features calculated by this code, with each feature averaged across all variables in the multivariate series.

• mean_TFB_characteristics_Exchange.csv: Contains the time series features used in the TFB paper, with each feature averaged across all variables in the multivariate series.

---

4. Introduction to TFB Features

For more details, please refer to here.

---

5. Code File

Click here to view the code.

TFB — [redacted] (□□)

---

1. [redacted]

• [redacted]conda[redacted], [redacted]python 3.8[redacted]R 4.3.1

```
conda create -n char_extractor python=3.8 r-base=4.3.1
conda activate char_extractor
```

• [redacted] R [redacted]

```
export R_HOME=$CONDA_PREFIX/lib/R
export PATH=$PATH:$R_HOME/bin
```

• [redacted] R [redacted]

```
conda install -c https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge r-tidyverse r-Rcatch22 r-forecast r-t
```

• [redacted]Python[redacted]

```
pip install rpy2==3.5.16 pandas==1.5.3 scipy==1.10.1 numpy==1.24.4 statsmodels==0.14.1 scikit_learn==1.3.2
```

---

2. [redacted]

• [redacted]

[redacted]TFB[redacted]3[redacted].

• [redacted]

• [redacted]./DemoDatasets/Exchange.csv.

• [redacted], [redacted]./DemoDatasets.

---

3. [redacted]

• [redacted]

• [redacted]: m4_hourly_dataset_69.csv[redacted]characteristics[redacted]2[redacted]

• All_characteristics_m4_hourly_dataset_69.csv: [redacted]

• TFB_characteristics_m4_hourly_dataset_69.csv: [redacted]TFB[redacted]

• [redacted]: Exchange.csv[redacted]characteristics[redacted]4[redacted]

• All_characteristics_Exchange.csv: [redacted], [redacted]

• TFB_characteristics_Exchange.csv: [redacted]TFB[redacted], [redacted]

• mean_All_characteristics_Exchange.csv: [redacted], [redacted]

• mean_TFB_characteristics_Exchange.csv: [redacted]TFB[redacted], [redacted]

4. TFB[redacted]

[redacted].

5. [redacted]

[redacted].